



Almacenamiento del Cliente

y APIs Multimedia desde JavaScript
Guia paso a paso para neofitos

Con IntelliJ IDEA y Node.js

localStorage • sessionStorage • IndexedDB
Video • Audio • Canvas • Geolocation • fetch()

Guayaquil, Ecuador • 2025

Para estudiantes que abren IntelliJ IDEA por primera vez

Índice

1. Herramientas y configuracion inicial	2
1.1. Instalar Node.js	2
1.2. Instalar y configurar IntelliJ IDEA	2
2. Parte I: Almacenamiento del Cliente	4
2.1. localStorage: datos que persisten para siempre	4
2.1.1. Ejercicio 1 – Guardar y recuperar preferencias de UI	4
2.2. sessionStorage: datos que duran la pestana	7
2.2.1. Ejercicio 2 – Formulario multipaso	7
2.3. IndexedDB: base de datos en el navegador	9
2.3.1. Ejercicio 3 – Agenda de cultivos offline	10
3. Parte II: APIs Multimedia desde JavaScript	14
3.1. Video y Audio: control desde JavaScript	14
3.1.1. Ejercicio 4 – Reproductor de video personalizado	14
3.2. Canvas: graficos desde JavaScript	17
3.2.1. Ejercicio 5 – Grafico de barras de cultivos	18
3.3. Geolocation API: posicion del usuario	21
3.3.1. Ejercicio 6 – Registrar parcela con GPS	21
3.4. fetch(): consumir APIs Multimedia	26
3.4.1. Ejercicio 7 – Subir y previsualizar imagenes	26
4. Resumen y buenas practicas	30
4.1. Cuando usar cada mecanismo de almacenamiento	30
4.2. Reglas de oro	31

1. Herramientas y configuracion inicial

Esta guia usa dos herramientas principales. **IntelliJ IDEA** es el entorno de desarrollo (IDE) donde escribiras el codigo. **Node.js** es el entorno que permite ejecutar JavaScript fuera del navegador y levantar un servidor local para que las APIs funcionen correctamente.

1.1. Instalar Node.js

Paso 1: Descargar Node.js

Abre tu navegador y ve a <https://nodejs.org>. Descarga la version **LTS** (Long Term Support), que es la mas estable. Al momento de escribir estas notas es la version 20.x.

- **Windows:** descarga el instalador .msi y ejecutalo. Acepta todas las opciones por defecto.
- **Mac:** descarga el paquete .pkg y siguelas instrucciones.
- **Linux Ubuntu/Debian:**

```
sudo apt update
sudo apt install nodejs npm
```

Paso 2: Verificar la instalacion

Abre una **terminal** (en Windows: busca “cmd” o “PowerShell” en el menu Inicio) y escribe:

```
node --version
npm --version
```

Debes ver algo como v20.11.0 y 10.2.4. Si aparece un numero de version, Node.js esta correctamente instalado.

1.2. Instalar y configurar IntelliJ IDEA

Paso 3: Descargar IntelliJ IDEA Community

Ve a <https://www.jetbrains.com/idea/download/>. Descarga la edicion **Community** (es gratuita). Instala con las opciones por defecto. Al abrir por primera vez veras la pantalla de bienvenida *Welcome to IntelliJ IDEA*.

Paso 4: Crear el proyecto de esta guia

En la pantalla de bienvenida:

1. Haz clic en **New Project**.
2. En el panel izquierdo selecciona **JavaScript and TypeScript** (si no aparece, selecciona **Empty Project**).
3. En *Project name* escribe: `guia-js-storage`.
4. Deja la ubicacion por defecto o elige una carpeta propia.
5. Haz clic en **Create**.

Paso 5: Estructura de carpetas del proyecto

Dentro del proyecto crea esta estructura. Para crear una carpeta: clic derecho sobre el nombre del proyecto en el panel izquierdo → *New* → *Directory*.

```
guia-js-storage/  
  partel-storage/  
    01-localStorage/  
    02-sessionStorage/  
    03-indexedDB/  
  parte2-multimedia/  
    04-video-audio/  
    05-canvas/  
    06-geolocation/  
    07-fetch-multimedia/
```

► Nota

En esta guia cada ejercicio es un archivo HTML que puedes abrir directamente en el navegador (doble clic sobre el archivo). No necesitas un servidor para `localStorage`, `sessionStorage` ni `Canvas`. Solo necesitaras un servidor local para **Geolocation** y `fetch()` con APIs propias (lo indicaremos en su momento).

2. Parte I: Almacenamiento del Cliente

El navegador ofrece tres mecanismos para guardar datos en el dispositivo del usuario. La **Tabla 1** los compara.

Mecanismo	Características	Cap.	API	Uso ideal
localStorage	Persistente, mismo origen, sincrónico, solo strings	5–10 MB	Sincrónica	Preferencias, carrito
sessionStorage	Solo la pestaña actual, mismo origen	5–10 MB	Sincrónica	Formulario multipaso
IndexedDB	Persistente, transaccional, asíncrono, objetos JS	GB	Asincrónica	Apps offline
Cookies	Enviadas con cada petición HTTP	4 KB	doc.cookie	Autenticación

Cuadro 1: APIs de almacenamiento del cliente en navegadores modernos.

2.1. localStorage: datos que persisten para siempre

localStorage guarda pares clave-valor que permanecen en el disco aunque el usuario cierre el navegador o apague el ordenador. Solo se borran si el usuario los borra manualmente o si el código llama a `removeItem` o `clear`.

Restricción clave: solo guarda *strings*. Para guardar objetos hay que convertirlos a JSON primero.

2.1.1. Ejercicio 1 – Guardar y recuperar preferencias de UI

IntelliJ IDEA — Paso 1: Crear el archivo

En IntelliJ, clic derecho sobre la carpeta `01-localStorage` → *New* → *HTML File*. Nombra el archivo `preferencias.html`. IntelliJ generará la estructura HTML básica automáticamente.

IntelliJ IDEA — Paso 2: Escribir el código

Borra el contenido generado y escribe exactamente lo siguiente. Observa que IntelliJ colorea el código y te muestra sugerencias mientras escribes (autocompletado).

Listing 1: preferencias.html

```

1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4 <meta charset="UTF-8">
```

```

5   <title>Preferencias con localStorage</title>
6   <style>
7     body { font-family: sans-serif; max-width: 500px;
8           margin: 40px auto; padding: 0 20px; }
9     button { padding: 8px 16px; margin: 4px;
10            cursor: pointer; }
11    #resultado { background: #f0f0f0; padding: 12px;
12               margin-top: 16px; border-radius: 6px;
13               font-family: monospace; }
14  </style>
15 </head>
16 <body>
17   <h1>Preferencias de usuario</h1>
18
19   <label>Tema:
20     <select id="tema">
21       <option value="claro">Claro</option>
22       <option value="oscuro">Oscuro</option>
23     </select>
24   </label>
25   <br><br>
26
27   <label>Idioma:
28     <select id="idioma">
29       <option value="es">Espanol</option>
30       <option value="en">English</option>
31     </select>
32   </label>
33   <br><br>
34
35   <button onclick="guardar()">Guardar preferencias</button>
36   <button onclick="leer()">Leer preferencias</button>
37   <button onclick="borrar()">Borrar preferencias</button>
38
39   <div id="resultado">Haz clic en un boton para comenzar.</div>
40
41   <script>
42     "use strict";
43
44     const CLAVE = "app:preferencias:v1";
45
46     // Al cargar la pagina, restaurar las preferencias guardadas
47     window.addEventListener("DOMContentLoaded", () => {
48       const guardado = localStorage.getItem(CLAVE);
49       if (guardado) {
50         const prefs = JSON.parse(guardado);
51         document.getElementById("tema").value = prefs.tema;
52         document.getElementById("idioma").value = prefs.idioma;
53         mostrar("Preferencias restauradas: " +
54               JSON.stringify(prefs));
55       }
56     });
57
58     function guardar() {
59       // Leer los valores del formulario
60       const prefs = {
61         tema: document.getElementById("tema").value,

```

```

62     idioma: document.getElementById("idioma").value
63 };
64
65 // Convertir a JSON y guardar
66 // (localStorage solo acepta strings)
67 try {
68     localStorage.setItem(CLAVE, JSON.stringify(prefs));
69     mostrar("Guardado correctamente: " +
70         JSON.stringify(prefs));
71 } catch (e) {
72     // El navegador lanza QuotaExceededError si esta lleno
73     mostrar("Error al guardar: " + e.message);
74 }
75 }
76
77 function leer() {
78     const raw = localStorage.getItem(CLAVE);
79     if (raw === null) {
80         mostrar("No hay preferencias guardadas.");
81     } else {
82         // Convertir de JSON a objeto JavaScript
83         const prefs = JSON.parse(raw);
84         mostrar("Preferencias leídas: " + JSON.stringify(prefs));
85     }
86 }
87
88 function borrar() {
89     localStorage.removeItem(CLAVE);
90     mostrar("Preferencias borradas.");
91 }
92
93 function mostrar(texto) {
94     document.getElementById("resultado").textContent = texto;
95 }
96 </script>
97 </body>
98 </html>

```

IntelliJ IDEA — Paso 3: Abrir en el navegador

En el panel de archivos de IntelliJ, haz clic derecho sobre `preferencias.html` → *Open in* → *Browser* → selecciona Chrome, Firefox o Edge. El navegador abrirá el archivo directamente.

IntelliJ IDEA — Paso 4: Probar en el navegador

1. Selecciona *Oscuro* en Tema y *English* en Idioma.
2. Haz clic en **Guardar preferencias**. El resultado debe mostrar el objeto guardado.
3. Cierra la pestaña del navegador y vuelve a abrirla (doble clic en el archivo desde IntelliJ).
4. Veras que los selectores ya tienen los valores que guardaste. `localStorage` persiste aunque cierres el navegador.

5. Abre las DevTools del navegador: presiona **F12** → pestaña *Application* → *Storage* → *localStorage*. Veras la clave y el valor almacenados.

✓ Resultado esperado en el navegador

Primer uso: el div muestra “Haz clic en un boton...”

Despues de guardar: Guardado correctamente:
{"tema":"oscuro","idioma":"en"}

Al recargar: los selectores muestran los valores guardados automaticamente.

► Importante

Nunca guardes contraseñas, tokens JWT ni datos sensibles en localStorage. JavaScript puede leer localStorage desde cualquier script de la pagina. Si hay un ataque XSS (inyeccion de codigo), el atacante roba todos esos datos. Para autenticacion usa **cookies con flag HttpOnly**.

2.2. sessionStorage: datos que duran la pestana

sessionStorage funciona exactamente igual que localStorage (mismos metodos, mismas restricciones), pero los datos se borran automaticamente cuando el usuario cierra la pestana. Es ideal para datos temporales de una sesion de trabajo, como los pasos de un formulario largo.

2.2.1. Ejercicio 2 – Formulario multipaso

IntelliJ IDEA — Paso 5: Crear el archivo

Crea 02-sessionStorage/formulario-multipaso.html en IntelliJ (mismo procedimiento que antes).

Listing 2: formulario-multipaso.html

```

1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <title>Formulario Multipaso</title>
6   <style>
7     body { font-family: sans-serif; max-width: 500px;
8           margin: 40px auto; padding: 0 20px; }
9     .paso { display: none; }
10    .paso.activo { display: block; }
11    input { display: block; width: 100%;
12          padding: 8px; margin: 8px 0;
13          box-sizing: border-box; }
14    button { padding: 8px 16px; margin: 4px; cursor: pointer; }
15    .progreso { color: #888; font-size: 13px; margin-bottom: 16px; }
16  </style>
17 </head>
18 <body>
19

```



```

20 <!-- PASO 1: Datos personales -->
21 <div id="paso1" class="paso activo">
22   <h2>Paso 1 de 3 -- Datos personales</h2>
23   <p class="progreso">Estos datos se guardan en sessionStorage
24     mientras navegas entre pasos.</p>
25   <input id="nombre" type="text" placeholder="Nombre completo">
26   <input id="email" type="email" placeholder="Correo electrónico">
27   <button onclick="irAPaso2()">Siguiete</button>
28 </div>
29
30 <!-- PASO 2: Datos del cultivo -->
31 <div id="paso2" class="paso">
32   <h2>Paso 2 de 3 -- Datos del cultivo</h2>
33   <input id="cultivo" type="text" placeholder="Nombre del cultivo">
34   <input id="area" type="number" placeholder="Area en hectareas">
35   <button onclick="irAPaso1()">Anterior</button>
36   <button onclick="irAPaso3()">Siguiete</button>
37 </div>
38
39 <!-- PASO 3: Revision y envio -->
40 <div id="paso3" class="paso">
41   <h2>Paso 3 de 3 -- Revision</h2>
42   <div id="resumen" style="background:#f5f5f5; padding:12px;
43     border-radius:6px; font-family:monospace;">
44   </div>
45   <br>
46   <button onclick="irAPaso2()">Anterior</button>
47   <button onclick="enviar()">Enviar</button>
48 </div>
49
50 <script>
51   "use strict";
52
53   const SS = sessionStorage; // alias para escribir menos
54
55   function irAPaso2() {
56     // Guardar datos del paso 1 antes de cambiar de pantalla
57     SS.setItem("paso1", JSON.stringify({
58       nombre: document.getElementById("nombre").value,
59       email: document.getElementById("email").value
60     }));
61     mostrarPaso("paso2");
62   }
63
64   function irAPaso3() {
65     // Guardar datos del paso 2
66     SS.setItem("paso2", JSON.stringify({
67       cultivo: document.getElementById("cultivo").value,
68       area: document.getElementById("area").value
69     }));
70     // Mostrar resumen con los datos de ambos pasos
71     const p1 = JSON.parse(SS.getItem("paso1") || "{}");
72     const p2 = JSON.parse(SS.getItem("paso2") || "{}");
73     document.getElementById("resumen").textContent =
74       JSON.stringify({ ...p1, ...p2 }, null, 2);
75     mostrarPaso("paso3");
76   }

```

```

77
78 function irAPaso1() { mostrarPaso("paso1"); }
79 function irAPaso2b(){ mostrarPaso("paso2"); }
80
81 function enviar() {
82     // Al terminar, limpiar sessionStorage
83     SS.removeItem("paso1");
84     SS.removeItem("paso2");
85     alert("Enviado con éxito. Datos borrados de sessionStorage.");
86     mostrarPaso("paso1");
87 }
88
89 function mostrarPaso(id) {
90     document.querySelectorAll(".paso")
91         .forEach(p => p.classList.remove("activo"));
92     document.getElementById(id).classList.add("activo");
93 }
94
95 // Restaurar datos si el usuario volvió atrás
96 window.addEventListener("DOMContentLoaded", () => {
97     const p1 = JSON.parse(SS.getItem("paso1") || "{}");
98     if (p1.nombre) document.getElementById("nombre").value = p1.nombre;
99     if (p1.email) document.getElementById("email").value = p1.email;
100 });
101 </script>
102 </body>
103 </html>

```

IntelliJ IDEA — Paso 6: Probar: la diferencia clave con localStorage

1. Rellena el Paso 1 y avanza al Paso 2.
2. Abre DevTools (F12) → *Application* → *sessionStorage*: veras los datos del Paso 1.
3. Abre una **nueva pestaña** y navega al mismo archivo. Los campos estarán vacíos. *sessionStorage* es exclusivo de cada pestaña.
4. Cierra y vuelve a abrir la pestaña original: los datos del Paso 1 han desaparecido. Esa es la diferencia con *localStorage*.

2.3. IndexedDB: base de datos en el navegador

IndexedDB es una base de datos transaccional completa dentro del navegador. A diferencia de *localStorage*, puede guardar objetos JavaScript directamente (sin convertirlos a string), manejar grandes volúmenes de datos, y crear índices para buscar eficientemente.

Característica principal: es *asíncrona*. Todas sus operaciones devuelven peticiones (IDBRequest) o se gestionan con Promises. Esto evita que la interfaz se bloquee mientras lee o escribe datos.

2.3.1. Ejercicio 3 – Agenda de cultivos offline

IntelliJ IDEA — Paso 7: Crear el archivo

Crea 03-indexedDB/agenda-cultivos.html.

Listing 3: agenda-cultivos.html (parte HTML)

```

1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <title>Agenda de Cultivos - IndexedDB</title>
6   <style>
7     body { font-family: sans-serif; max-width: 600px;
8           margin: 40px auto; padding: 0 20px; }
9     input, select { padding: 6px; margin: 4px;
10                  width: 180px; }
11    button { padding: 8px 14px; cursor: pointer; margin: 4px; }
12    table { width: 100%; border-collapse: collapse;
13          margin-top: 16px; }
14    th, td { border: 1px solid #ddd; padding: 8px;
15            text-align: left; }
16    th { background: #f2f2f2; }
17    #msg { color: #1D9E75; font-weight: bold;
18          margin: 8px 0; }
19  </style>
20 </head>
21 <body>
22   <h1>Agenda de Cultivos</h1>
23   <p>Los datos se guardan en IndexedDB -- persisten sin servidor.</p>
24
25   <div>
26     <input id="nombre" type="text" placeholder="Cultivo">
27     <input id="area" type="number" placeholder="Hectareas">
28     <select id="estado">
29       <option value="germinacion">Germinacion</option>
30       <option value="crecimiento">Crecimiento</option>
31       <option value="cosechado">Cosechado</option>
32     </select>
33     <button onclick="agregarCultivo()">Agregar</button>
34   </div>
35
36   <p id="msg"></p>
37
38   <table>
39     <thead>
40       <tr><th>ID</th><th>Cultivo</th><th>Ha</th>
41         <th>Estado</th><th>Accion</th></tr>
42     </thead>
43     <tbody id="tabla"></tbody>
44   </table>
45
46   <script>
47     "use strict";
48
49     // ---- 1. ABRIR / CREAR LA BASE DE DATOS -----

```

```

50 // indexedDB.open(nombre, numero_de_version)
51 // La version sube cuando cambias la estructura (como una migracion)
52 const peticion = indexedDB.open("AgriDB", 1);
53
54 // onupgradeneeded: se ejecuta la primera vez o cuando sube la version
55 peticion.onupgradeneeded = (evento) => {
56   const db = evento.target.result;
57   // Crear el "object store" (equivale a una tabla)
58   // keyPath: campo que actua como clave primaria
59   const store = db.createObjectStore("cultivos", {
60     keyPath: "id",
61     autoIncrement: true // el ID se genera solo
62   });
63   // Crear un indice para poder buscar por nombre
64   store.createIndex("por_nombre", "nombre", { unique: false });
65   mensaje("Base de datos creada.");
66 };
67
68 // onsuccess: se ejecuta cuando la BD ya esta lista
69 let db;
70 peticion.onsuccess = (evento) => {
71   db = evento.target.result;
72   mensaje("BD lista. Cargando registros...");
73   cargarTodos();
74 };
75
76 peticion.onerror = (evento) => {
77   mensaje("Error abriendo BD: " + evento.target.error);
78 };
79
80 // ---- 2. AGREGAR UN REGISTRO -----
81 function agregarCultivo() {
82   const cultivo = {
83     nombre: document.getElementById("nombre").value.trim(),
84     area:   parseFloat(document.getElementById("area").value) || 0,
85     estado: document.getElementById("estado").value,
86     fecha:  new Date().toLocaleDateString("es-EC")
87   };
88   if (!cultivo.nombre) {
89     mensaje("Escribe el nombre del cultivo.");
90     return;
91   }
92
93   // Abrir una transaccion de escritura ("readwrite")
94   const tx = db.transaction("cultivos", "readwrite");
95   const store = tx.objectStore("cultivos");
96   const req = store.add(cultivo); // agregar el objeto
97
98   req.onsuccess = () => {
99     mensaje("Cultivo guardado con id: " + req.result);
100     cargarTodos();
101     // Limpiar campos
102     document.getElementById("nombre").value = "";
103     document.getElementById("area").value = "";
104   };
105   req.onerror = () => mensaje("Error al guardar.");
106 }

```

```

107
108 // ---- 3. LEER TODOS LOS REGISTROS -----
109 function cargarTodos() {
110     // Transaccion de solo lectura ("readonly")
111     const tx = db.transaction("cultivos", "readonly");
112     const store = tx.objectStore("cultivos");
113     const req = store.getAll(); // traer todos los registros
114
115     req.onsuccess = () => {
116         const cultivos = req.result;
117         const tbody = document.getElementById("tabla");
118         tbody.innerHTML = "";
119
120         cultivos.forEach(c => {
121             const fila = document.createElement("tr");
122             fila.innerHTML =
123                 '<td>' + c.id + '</td>' +
124                 '<td>' + c.nombre + '</td>' +
125                 '<td>' + c.area + '</td>' +
126                 '<td>' + c.estado + '</td>' +
127                 '<td><button onclick="eliminar(' + c.id + ')">' +
128                 'Borrar</button></td>';
129             tbody.appendChild(fila);
130         });
131         mensaje("Registros cargados: " + cultivos.length);
132     };
133 }
134
135 // ---- 4. ELIMINAR UN REGISTRO -----
136 function eliminar(id) {
137     const tx = db.transaction("cultivos", "readwrite");
138     const store = tx.objectStore("cultivos");
139     store.delete(id).onsuccess = () => {
140         mensaje("Cultivo " + id + " eliminado.");
141         cargarTodos();
142     };
143 }
144
145 function mensaje(texto) {
146     document.getElementById("msg").textContent = texto;
147 }
148 </script>
149 </body>
150 </html>

```

IntelliJ IDEA — Paso 8: Probar en el navegador

1. Abre el archivo. La primera vez vera el mensaje “Base de datos creada”.
2. Agrega 3 o 4 cultivos. Aparecen en la tabla.
3. Cierra la pestana y vuelve a abrirla. Los cultivos siguen ahi (IndexedDB es persistente).
4. Abre DevTools → *Application* → *IndexedDB* → *AgriDB* → *cultivos*. Veras los registros almacenados como objetos JavaScript.

5. Haz clic en *Borrar* para eliminar un registro.

3. Parte II: APIs Multimedia desde JavaScript

HTML5 incluye cinco APIs nativas que reemplazaron tecnologías propietarias como Flash: `<video>` y `<audio>`, `<canvas>`, Geolocation API, Web Storage (que ya vimos), y Drag-and-Drop API. En esta parte estudiaremos las cuatro primeras con ejemplos paso a paso.

3.1. Video y Audio: control desde JavaScript

Los elementos `<video>` y `<audio>` exponen una API JavaScript completa para reproducir, pausar, ajustar el volumen, saltar a un momento específico y reaccionar a eventos del ciclo de vida de la reproducción.

3.1.1. Ejercicio 4 – Reproductor de video personalizado

IntelliJ IDEA — Paso 1: Preparar un video de prueba

Para este ejercicio necesitas un archivo de video. Puedes:

- Descargar cualquier video `.mp4` de tu ordenador y copiarlo a la carpeta `04-video-audio/`.
- O usar la URL de un video publico en la red (para esto si necesitaras conexion a internet).

En el codigo usaremos `sample.mp4` como nombre. Renombra tu archivo o ajusta el `src` en el codigo.

IntelliJ IDEA — Paso 2: Crear el archivo

Crea `04-video-audio/reproductor.html`.

Listing 4: reproductor.html

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <title>Reproductor de Video</title>
6   <style>
7     body { font-family: sans-serif; max-width: 660px;
8           margin: 40px auto; padding: 0 20px; }
9     video { width: 100%; display: block; background: #000; }
10    .controles { display: flex; gap: 8px; margin: 8px 0;
11               flex-wrap: wrap; align-items: center; }
12    button { padding: 6px 14px; cursor: pointer; }
13    input[type="range"] { flex: 1; min-width: 100px; }
14    #info { font-size: 13px; color: #666;
15           font-family: monospace; margin-top: 6px; }
16  </style>
17 </head>
18 <body>
19   <h1>Reproductor Personalizado</h1>
```

```

20
21 <!-- El atributo controls muestra los controles nativos del navegador -->
22 <!-- preload="metadata": solo descarga la duracion y dimensiones -->
23 <!-- poster: imagen que se ve antes de reproducir -->
24 <figure>
25   <video id="miVideo"
26     preload="metadata"
27     poster="poster.jpg">
28     <!-- El navegador usa el PRIMERO que puede reproducir -->
29     <source src="sample.webm" type="video/webm">
30     <source src="sample.mp4" type="video/mp4">
31     <!-- Subtitulos accesibles (archivo WebVTT) -->
32     <track kind="captions"
33       src="subtitulos.es.vtt"
34       srclang="es"
35       label="Espanol"
36       default>
37     <!-- Mensaje para navegadores muy antiguos -->
38     <p>Tu navegador no soporta video HTML5.
39     <a href="sample.mp4">Descargar el video</a>.</p>
40   </video>
41   <figcaption>Video de demostracion -- Guia JS 2025</figcaption>
42 </figure>
43
44 <!-- Controles personalizados -->
45 <div class="controles">
46   <button id="btnPlay" onclick="togglePlay()">Reproducir</button>
47   <button onclick="retroceder()">-10s</button>
48   <button onclick="avanzar()">+10s</button>
49   <label>Volumen:
50     <input type="range" id="volumen"
51       min="0" max="1" step="0.1" value="1"
52       oninput="ajustarVolumen(this.value)">
53   </label>
54   <label>Velocidad:
55     <select onchange="ajustarVelocidad(this.value)">
56       <option value="0.5">0.5x</option>
57       <option value="1" selected>1x</option>
58       <option value="1.5">1.5x</option>
59       <option value="2">2x</option>
60     </select>
61   </label>
62   <button onclick="document.getElementById('miVideo')
63     .requestFullscreen()">
64     Pantalla completa
65   </button>
66 </div>
67
68 <!-- Barra de progreso manual -->
69 <input type="range" id="progreso"
70   min="0" max="100" value="0" step="0.1"
71   style="width:100%; margin:4px 0"
72   oninput="saltar(this.value)">
73
74 <div id="info">Haz clic en Reproducir para comenzar.</div>
75
76 <script>

```



```
77     "use strict";
78
79     const video = document.getElementById("miVideo");
80
81     // ---- CONTROLES -----
82
83     function togglePlay() {
84         if (video.paused) {
85             video.play();
86             document.getElementById("btnPlay").textContent = "Pausar";
87         } else {
88             video.pause();
89             document.getElementById("btnPlay").textContent = "Reproducir";
90         }
91     }
92
93     function retroceder() {
94         video.currentTime = Math.max(0, video.currentTime - 10);
95     }
96
97     function avanzar() {
98         video.currentTime = Math.min(video.duration,
99                                     video.currentTime + 10);
100    }
101
102    function ajustarVolumen(val) {
103        video.volume = parseFloat(val);
104    }
105
106    function ajustarVelocidad(val) {
107        video.playbackRate = parseFloat(val);
108    }
109
110    function saltar(porcentaje) {
111        video.currentTime = (porcentaje / 100) * video.duration;
112    }
113
114    // ---- EVENTOS DEL VIDEO -----
115
116    // timeupdate: se dispara continuamente mientras reproduce
117    video.addEventListener("timeupdate", () => {
118        const seg = Math.floor(video.currentTime);
119        const total = Math.floor(video.duration || 0);
120        const pct = total > 0
121            ? (video.currentTime / video.duration) * 100
122            : 0;
123        document.getElementById("progreso").value =
124            pct.toFixed(1);
125        document.getElementById("info").textContent =
126            "Tiempo: " + seg + "s / " + total + "s" +
127            " | Volumen: " + (video.volume * 100).toFixed(0) + "%" +
128            " | Velocidad: " + video.playbackRate + "x";
129    });
130
131    video.addEventListener("ended", () => {
132        document.getElementById("btnPlay").textContent = "Reproducir";
133        document.getElementById("info").textContent = "Video terminado.";
```

```

134     });
135
136     video.addEventListener("error", () => {
137         document.getElementById("info").textContent =
138             "Error: no se pudo cargar el video. " +
139             "Verifica que sample.mp4 este en la misma carpeta.";
140     });
141 </script>
142 </body>
143 </html>

```

IntelliJ IDEA — Paso 3: Probar en el navegador

1. Abre `reproductor.html`. Si no tienes `sample.mp4`, el area del video estara negra y el `id="info"` mostrara el mensaje de error. Copia cualquier `.mp4` en la carpeta y refresca.
2. Haz clic en **Reproducir**. El boton cambia a "Pausar".
3. Mueve el deslizador de progreso para saltar a otro momento del video.
4. Cambia la velocidad a **1.5x**. El video se reproduce mas rapido.
5. Baja el volumen con el deslizador.
6. En DevTools (F12) → *Console*, escribe `video.duration` y presiona Enter. Veras la duracion en segundos.

► Nota

El archivo WebVTT de subtítulos. Si quieres probar los subtítulos, crea el archivo `subtitulos.es.vtt` en la misma carpeta con este contenido:

```

WEBVTT

00:00:01.000 --> 00:00:04.000
Bienvenidos a la guia de JavaScript.

00:00:04.500 --> 00:00:08.000
Hoy aprenderemos sobre APIs del navegador.

```

Los tiempos siguen el formato `HH:MM:SS.mmm` → `HH:MM:SS.mmm`.

3.2. Canvas: graficos desde JavaScript

El elemento `<canvas>` es un lienzo de dibujo en el que JavaScript puede pintar formas, texto, imagenes y animaciones utilizando la API de contexto 2D. Es la base de graficas, juegos y visualizaciones de datos en el navegador.

3.2.1. Ejercicio 5 – Grafico de barras de cultivos

IntelliJ IDEA — Paso 4: Crear el archivo

Crea 05-canvas/grafico-barras.html.

Listing 5: grafico-barras.html

```

1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <title>Grafico de Barras con Canvas</title>
6   <style>
7     body { font-family: sans-serif; max-width: 720px;
8           margin: 40px auto; padding: 0 20px; }
9     canvas { border: 1px solid #ddd; display: block;
10            margin-top: 16px; }
11    button { padding: 8px 16px; cursor: pointer; margin: 4px; }
12  </style>
13 </head>
14 <body>
15   <h1>Produccion de Cultivos (hectareas)</h1>
16   <button onclick="dibujar()">Dibujar grafico</button>
17   <button onclick="limpiar()">Limpiar</button>
18   <button onclick="exportar()">Exportar como PNG</button>
19
20   <!-- El canvas necesita ancho y alto declarados aqui en el HTML -->
21   <!-- Si los pones en CSS se distorsiona la escala de dibujo -->
22   <canvas id="miCanvas" width="680" height="380"></canvas>
23
24   <script>
25     "use strict";
26
27     // Obtener el contexto 2D: todas las funciones de dibujo vienen de aqui
28     const canvas = document.getElementById("miCanvas");
29     const ctx = canvas.getContext("2d");
30
31     // Datos de ejemplo (podrian venir de una API)
32     const datos = [
33       { cultivo: "Maiz", hectareas: 3.5, color: "#1D9E75" },
34       { cultivo: "Arroz", hectareas: 5.0, color: "#185FA5" },
35       { cultivo: "Cacao", hectareas: 4.0, color: "#BA7517" },
36       { cultivo: "Banano", hectareas: 2.0, color: "#A32D2D" },
37       { cultivo: "Tomate", hectareas: 1.0, color: "#7B2FBE" }
38     ];
39
40     function dibujar() {
41       // --- Configuracion del grafico ---
42       const margenIzq = 60; // espacio para etiquetas del eje Y
43       const margenInf = 50; // espacio para etiquetas del eje X
44       const margenSup = 30; // espacio superior
45       const anchoTotal = canvas.width - margenIzq - 20;
46       const altoTotal = canvas.height - margenInf - margenSup;
47
48       // Limpiar antes de dibujar
49       ctx.clearRect(0, 0, canvas.width, canvas.height);

```

```
50
51 // --- Titulo ---
52 ctx.fillStyle = "#2C2C2A";
53 ctx.font      = "bold 16px sans-serif";
54 ctx.textAlign = "center";
55 ctx.fillText(
56     "Hectareas por cultivo -- AgriApp 2025",
57     canvas.width / 2, 20
58 );
59
60 // --- Calculos de escala ---
61 const maxVal = Math.max(...datos.map(d => d.hectareas));
62 const anchoBar = anchoTotal / datos.length - 20;
63
64 // --- Dibujar ejes ---
65 ctx.strokeStyle = "#888";
66 ctx.lineWidth   = 1;
67 ctx.beginPath();
68 // Eje Y (vertical)
69 ctx.moveTo(margenIzq, margenSup);
70 ctx.lineTo(margenIzq, canvas.height - margenInf);
71 // Eje X (horizontal)
72 ctx.lineTo(canvas.width - 10, canvas.height - margenInf);
73 ctx.stroke();
74
75 // --- Lineas horizontales de referencia ---
76 ctx.strokeStyle = "#e0e0e0";
77 ctx.setLineDash([4, 4]); // linea punteada
78 for (let i = 1; i <= 5; i++) {
79     const y = (canvas.height - margenInf) -
80             (altoTotal / 5) * i;
81     ctx.beginPath();
82     ctx.moveTo(margenIzq, y);
83     ctx.lineTo(canvas.width - 10, y);
84     ctx.stroke();
85     // Etiqueta del valor
86     ctx.setLineDash([]); // volver a linea solida
87     ctx.fillStyle = "#888";
88     ctx.font      = "11px sans-serif";
89     ctx.textAlign = "right";
90     ctx.fillText(
91         ((maxVal / 5) * i).toFixed(1) + " ha",
92         margenIzq - 5, y + 4
93     );
94     ctx.setLineDash([4, 4]);
95 }
96 ctx.setLineDash([]); // restaurar linea solida
97
98 // --- Dibujar las barras ---
99 datos.forEach((d, i) => {
100     const altoBarra = (d.hectareas / maxVal) * altoTotal;
101     const x         = margenIzq + 10 + i * (anchoBar + 20);
102     const y         = canvas.height - margenInf - altoBarra;
103
104     // Barra coloreada
105     ctx.fillStyle = d.color;
106     ctx.fillRect(x, y, anchoBar, altoBarra);
```

```
107
108 // Borde de la barra
109 ctx.strokeStyle = d.color;
110 ctx.lineWidth = 1;
111 ctx.strokeRect(x, y, anchoBar, altoBarra);
112
113 // Valor encima de la barra
114 ctx.fillStyle = "#2C2C2A";
115 ctx.font = "bold 13px sans-serif";
116 ctx.textAlign = "center";
117 ctx.fillText(
118     d.hectareas + " ha",
119     x + anchoBar / 2, y - 6
120 );
121
122 // Nombre del cultivo debajo del eje X
123 ctx.fillStyle = "#444";
124 ctx.font = "12px sans-serif";
125 ctx.fillText(
126     d.cultivo,
127     x + anchoBar / 2,
128     canvas.height - margenInf + 20
129 );
130 });
131 }
132
133 function limpiar() {
134     ctx.clearRect(0, 0, canvas.width, canvas.height);
135 }
136
137 function exportar() {
138     // toDataURL convierte el canvas a una imagen PNG en base64
139     const enlace = document.createElement("a");
140     enlace.download = "grafico-cultivos.png";
141     enlace.href = canvas.toDataURL("image/png");
142     enlace.click(); // dispara la descarga
143 }
144
145 // Dibujar automaticamente al cargar la pagina
146 window.addEventListener("DOMContentLoaded", dibujar);
147 </script>
148 </body>
149 </html>
```

IntelliJ IDEA — Paso 5: Probar el grafico

1. Abre el archivo. El grafico aparece automaticamente.
2. Haz clic en **Exportar como PNG**: el navegador descargara el grafico como imagen.
3. Modifica los valores en el array **datos** (por ejemplo cambia hectareas: 3.5 a hectareas: 7.0), guarda el archivo en IntelliJ y recarga el navegador.

3.3. Geolocation API: posicion del usuario

La API de Geolocation permite obtener las coordenadas GPS del usuario. Requiere que el usuario acepte el permiso explícitamente y funciona mejor con HTTPS (en producción). En localhost funciona sin HTTPS.

3.3.1. Ejercicio 6 – Registrar parcela con GPS

► Nota

Para este ejercicio necesitamos un servidor local porque el ejemplo hace una petición fetch a una API propia. Levantaremos un servidor mínimo con Node.js.

IntelliJ IDEA — Paso 6: Crear la estructura del servidor

En IntelliJ, crea esta estructura dentro de 06-geolocation/:

```
06-geolocation/
  server.js      (servidor Node.js)
  public/
    index.html   (la pagina web)
```

IntelliJ IDEA — Paso 7: Crear el servidor Node.js

Crea 06-geolocation/server.js:

Listing 6: server.js – servidor mínimo con Node.js

```
1  "use strict";
2
3  const http = require("http");
4  const fs   = require("fs");
5  const path = require("path");
6
7  const parcelas = []; // "base de datos" en memoria
8
9  const server = http.createServer((req, res) => {
10
11    // Servir el archivo HTML
12    if (req.method === "GET" && req.url === "/") {
13      const html = fs.readFileSync(
14        path.join(__dirname, "public", "index.html")
15      );
16      res.writeHead(200, { "Content-Type": "text/html" });
17      res.end(html);
18      return;
19    }
20
21    // POST /api/parcelas: recibir coordenadas
22    if (req.method === "POST" && req.url === "/api/parcelas") {
23      let body = "";
24      req.on("data", chunk => { body += chunk; });
25      req.on("end", () => {
```

```
26     try {
27         const datos = JSON.parse(body);
28         datos.id = parcelas.length + 1;
29         parcelas.push(datos);
30         console.log("Parcela recibida:", datos);
31
32         res.writeHead(201, { "Content-Type": "application/json" });
33         res.end(JSON.stringify({
34             mensaje: "Parcela registrada",
35             parcela: datos
36         }));
37     } catch (e) {
38         res.writeHead(400);
39         res.end(JSON.stringify({ error: "JSON invalido" }));
40     }
41 });
42 return;
43 }
44
45 // GET /api/parcelas: devolver todas las parcelas
46 if (req.method === "GET" && req.url === "/api/parcelas") {
47     res.writeHead(200, { "Content-Type": "application/json" });
48     res.end(JSON.stringify({ total: parcelas.length, parcelas }));
49     return;
50 }
51
52 res.writeHead(404);
53 res.end("No encontrado");
54 });
55
56 server.listen(3000, () => {
57     console.log("Servidor en http://localhost:3000");
58 });
```

IntelliJ IDEA — Paso 8: Crear la pagina HTML

Crea 06-geolocation/public/index.html:

Listing 7: public/index.html

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4     <meta charset="UTF-8">
5     <title>Registro de Parcelas con GPS</title>
6     <style>
7         body { font-family: sans-serif; max-width: 500px;
8             margin: 40px auto; padding: 0 20px; }
9         input { display: block; width: 100%; padding: 8px;
10             margin: 6px 0; box-sizing: border-box; }
11         button { padding: 8px 16px; cursor: pointer; margin: 4px; }
12         #coords { font-family: monospace; background: #f5f5f5;
13             padding: 10px; border-radius: 6px;
14             margin: 12px 0; }
15         #lista { margin-top: 16px; }
16         .item { border: 1px solid #ddd; padding: 8px;
```

```

17         border-radius: 4px; margin: 6px 0;
18         font-size: 13px; }
19     .error { color: red; }
20     .ok     { color: #1D9E75; }
21 </style>
22 </head>
23 <body>
24     <h1>Registro de Parcelas GPS</h1>
25
26     <input id="nombre" type="text" placeholder="Nombre de la parcela">
27
28     <button onclick="obtenerUbicacion()">
29         Obtener mi ubicacion GPS
30     </button>
31
32     <div id="coords">Haz clic en el boton para obtener las coordenadas.</div>
33
34     <button id="btnRegistrar" onclick="registrar()" disabled>
35         Registrar parcela
36     </button>
37     <button onclick="cargarTodas()">Ver todas las parcelas</button>
38
39     <div id="resultado"></div>
40     <div id="lista"></div>
41
42     <script>
43         "use strict";
44
45         let latActual = null;
46         let lonActual = null;
47
48         // ---- 1. OBTENER UBICACION GPS -----
49         function obtenerUbicacion() {
50             // Verificar que el navegador soporta Geolocation
51             if (!("geolocation" in navigator)) {
52                 mostrarError("Tu navegador no soporta Geolocation.");
53                 return;
54             }
55
56             document.getElementById("coords").textContent =
57                 "Obteniendo ubicacion... (acepta el permiso)";
58
59             navigator.geolocation.getCurrentPosition(
60                 // Funcion de exito
61                 (posicion) => {
62                     latActual = posicion.coords.latitude;
63                     lonActual = posicion.coords.longitude;
64                     const precision = posicion.coords.accuracy;
65
66                     document.getElementById("coords").innerHTML =
67                         "<strong>Latitud:</strong> " + latActual.toFixed(6) +
68                         "<br><strong>Longitud:</strong> " + lonActual.toFixed(6) +
69                         "<br><strong>Precision:</strong> " +
70                         precision.toFixed(0) + " metros";
71
72                     // Habilitar el boton de registrar
73                     document.getElementById("btnRegistrar").disabled = false;

```



```

74     mostrarOk("Coordenadas obtenidas. Ya puedes registrar.");
75 },
76 // Funcion de error
77 (error) => {
78     let msg;
79     switch (error.code) {
80         case 1: msg = "Permiso denegado por el usuario."; break;
81         case 2: msg = "Posicion no disponible."; break;
82         case 3: msg = "Tiempo de espera agotado."; break;
83         default: msg = "Error desconocido: " + error.message;
84     }
85     mostrarError(msg);
86 },
87 // Opciones: alta precision, maximo 10 segundos
88 { enableHighAccuracy: true, timeout: 10000, maximumAge: 0 }
89 );
90 }
91
92 // ---- 2. ENVIAR COORDENADAS AL SERVIDOR -----
93 async function registrar() {
94     const nombre = document.getElementById("nombre").value.trim();
95     if (!nombre) {
96         mostrarError("Escribe el nombre de la parcela.");
97         return;
98     }
99     if (latActual === null) {
100         mostrarError("Primero obtiene la ubicacion GPS.");
101         return;
102     }
103
104     try {
105         const respuesta = await fetch("/api/parcelas", {
106             method: "POST",
107             headers: { "Content-Type": "application/json" },
108             body: JSON.stringify({
109                 nombre,
110                 latitud: latActual,
111                 longitud: lonActual,
112                 fecha: new Date().toLocaleString("es-EC")
113             })
114         });
115
116         if (!respuesta.ok)
117             throw new Error("HTTP " + respuesta.status);
118
119         const datos = await respuesta.json();
120         mostrarOk("Registrada: " + JSON.stringify(datos.parcela));
121         document.getElementById("nombre").value = "";
122     } catch (e) {
123         mostrarError("Error al registrar: " + e.message);
124     }
125 }
126
127 // ---- 3. CARGAR TODAS LAS PARCELAS -----
128 async function cargarTodas() {
129     try {
130         const resp = await fetch("/api/parcelas");

```

```

131     const datos = await resp.json();
132     const lista = document.getElementById("lista");
133     lista.innerHTML = "<h3>Parcelas registradas ("
134                     + datos.total + "):</h3>";
135     datos.parcelas.forEach(p => {
136         const div = document.createElement("div");
137         div.className = "item";
138         div.textContent = "[" + p.id + "]" + p.nombre +
139                         " | Lat: " + p.latitud.toFixed(5) +
140                         " | Lon: " + p.longitud.toFixed(5) +
141                         " | " + p.fecha;
142         lista.appendChild(div);
143     });
144     } catch (e) {
145         mostrarError("Error cargando parcelas: " + e.message);
146     }
147 }
148
149 function mostrarOk(msg) {
150     document.getElementById("resultado").className = "ok";
151     document.getElementById("resultado").textContent = msg;
152 }
153 function mostrarError(msg) {
154     document.getElementById("resultado").className = "error";
155     document.getElementById("resultado").textContent = msg;
156 }
157 </script>
158 </body>
159 </html>

```

IntelliJ IDEA — Paso 9: Levantar el servidor desde IntelliJ

En IntelliJ, abre una **Terminal** integrada: *View* → *Tool Windows* → *Terminal* (o presiona **Alt+F12**). Navega a la carpeta del ejercicio y ejecuta:

```
cd 06-geolocation
node server.js
```

Veras: Servidor en <http://localhost:3000>. Ahora abre <http://localhost:3000> en el navegador.

IntelliJ IDEA — Paso 10: Probar la aplicacion

1. Haz clic en **Obtener mi ubicacion GPS**. El navegador preguntara si deseas permitir el acceso a la ubicacion. Haz clic en *Permitir*.
2. Veras las coordenadas (en un ordenador de escritorio la precision puede ser de varios cientos de metros; en un telefono movil es mucho mas precisa).
3. Escribe un nombre de parcela y haz clic en **Registrar parcela**. El servidor recibira los datos via *fetch*.
4. Haz clic en **Ver todas las parcelas** para confirmar que se guardaron.

3.4. fetch(): consumir APIs Multimedia

`fetch()` es la función nativa de JavaScript para hacer peticiones HTTP. Devuelve una **Promesa** (resultado futuro), por lo que se usa con `async/await` para escribir código asíncrono de forma clara. En esta sección la combinamos con `FileReader` para subir y previsualizar archivos multimedia.

3.4.1. Ejercicio 7 – Subir y previsualizar imágenes

IntelliJ IDEA — Paso 11: Crear la estructura

Dentro de `07-fetch-multimedia/` crea: `server.js` y la carpeta `public/` con `index.html` y la subcarpeta `subidas/`.

IntelliJ IDEA — Paso 12: Servidor que acepta archivos

Crea `07-fetch-multimedia/server.js`:

Listing 8: `server.js` para subida de archivos

```
1  "use strict";
2
3  const http = require("http");
4  const fs   = require("fs");
5  const path = require("path");
6
7  // Instalar: npm install formidable
8  const formidable = require("formidable");
9
10 const DIRECTORIO_SUBIDAS = path.join(__dirname, "subidas");
11 if (!fs.existsSync(DIRECTORIO_SUBIDAS))
12   fs.mkdirSync(DIRECTORIO_SUBIDAS);
13
14 function servirArchivo(res, rutaRelativa, tipoMime) {
15   try {
16     const contenido = fs.readFileSync(
17       path.join(__dirname, "public", rutaRelativa)
18     );
19     res.writeHead(200, { "Content-Type": tipoMime });
20     res.end(contenido);
21   } catch (e) {
22     res.writeHead(404);
23     res.end("No encontrado: " + rutaRelativa);
24   }
25 }
26
27 const server = http.createServer((req, res) => {
28   // Cabeceras CORS para permitir fetch desde el navegador
29   res.setHeader("Access-Control-Allow-Origin", "*");
30
31   if (req.method === "GET" && req.url === "/") {
32     servirArchivo(res, "index.html", "text/html");
33     return;
34   }
35 }
```

```
36 // Recibir la imagen subida con FormData
37 if (req.method === "POST" && req.url === "/api/subir") {
38   const form = formidable({
39     uploadDir: DIRECTORIO_SUBIDAS,
40     keepExtensions: true,
41     maxFileSize: 5 * 1024 * 1024 // 5 MB maximo
42   });
43
44   form.parse(req, (err, fields, files) => {
45     if (err) {
46       res.writeHead(400, { "Content-Type": "application/json" });
47       res.end(JSON.stringify({ error: err.message }));
48       return;
49     }
50
51     const archivo = files.imagen?.[0] || files.imagen;
52     if (!archivo) {
53       res.writeHead(400, { "Content-Type": "application/json" });
54       res.end(JSON.stringify({ error: "No se recibio imagen" }));
55       return;
56     }
57
58     const nombre = path.basename(archivo.filepath);
59     console.log("Imagen recibida:", nombre,
60               archivo.size, "bytes");
61
62     res.writeHead(200, { "Content-Type": "application/json" });
63     res.end(JSON.stringify({
64       mensaje: "Imagen guardada",
65       nombre,
66       tamano: archivo.size,
67       tipo: archivo.mimetype
68     }));
69   });
70   return;
71 }
72
73 res.writeHead(404); res.end();
74 });
75
76 server.listen(3000, () => {
77   console.log("Servidor en http://localhost:3000");
78   console.log("Instala formidable si no lo tienes:");
79   console.log(" npm install formidable");
80 });
```

IntelliJ IDEA — Paso 13: Instalar la dependencia formidable

En la Terminal de IntelliJ (**Alt+F12**):

```
cd 07-fetch-multimedia
npm init -y
npm install formidable
```

Listing 9: public/index.html – previsualizar y subir imagen

```

1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <title>Subir Imagen con fetch()</title>
6   <style>
7     body { font-family: sans-serif; max-width: 520px;
8           margin: 40px auto; padding: 0 20px; }
9     #preview{ max-width: 100%; max-height: 280px;
10             display: none; margin: 12px 0;
11             border: 2px solid #ddd; border-radius: 8px; }
12     button { padding: 8px 16px; cursor: pointer; margin: 4px; }
13     #info { font-family: monospace; font-size: 13px;
14            background: #f5f5f5; padding: 10px;
15            border-radius: 6px; margin-top: 12px; }
16     .ok { color: #1D9E75; }
17     .error { color: red; }
18   </style>
19 </head>
20 <body>
21   <h1>Subir imagen al servidor</h1>
22
23   <!-- accept: filtra los tipos de archivo en el dialogo -->
24   <input type="file" id="archivo" accept="image/*">
25   <br>
26
27   <!-- Imagen de previsualizacion (oculta hasta que se seleccione) -->
28   <img id="preview" alt="Previsualizacion de la imagen">
29
30   <div id="info">Selecciona una imagen para comenzar.</div>
31
32   <button id="btnSubir" onclick="subirImagen()" disabled>
33     Subir al servidor
34   </button>
35
36   <script>
37     "use strict";
38
39     const inputArchivo = document.getElementById("archivo");
40     const preview      = document.getElementById("preview");
41     const info         = document.getElementById("info");
42     const btnSubir     = document.getElementById("btnSubir");
43
44     // ---- 1. PREVISUALIZAR SIN SUBIR (FileReader) -----
45     inputArchivo.addEventListener("change", (e) => {
46       const archivo = e.target.files[0];
47
48       if (!archivo) {
49         mostrar("No se selecciono archivo.", "error");
50         return;
51       }
52
53       // Mostrar informacion del archivo local
54       mostrar(
55         "Archivo: " + archivo.name + "\n" +

```

```
56     "Tipo: "      + archivo.type      + "\n" +
57     "Tamano: "    + (archivo.size / 1024).toFixed(1) + " KB\n" +
58     "Modificado: " + new Date(archivo.lastModified)
59                   .toLocaleString("es-EC"),
60     "ok"
61 );
62
63 // Metodo 1: URL.createObjectURL (mas eficiente)
64 // Crea una URL temporal que apunta al archivo local
65 preview.src = URL.createObjectURL(archivo);
66 preview.style.display = "block";
67
68 // Habilitar el boton de subida
69 btnSubir.disabled = false;
70 });
71
72 // ---- 2. SUBIR AL SERVIDOR CON fetch() + FormData ----
73 async function subirImagen() {
74     const archivo = inputArchivo.files[0];
75     if (!archivo) return;
76
77     btnSubir.textContent = "Subiendo...";
78     btnSubir.disabled    = true;
79
80     // FormData permite enviar archivos binarios en una peticion POST
81     // NO pongas Content-Type manualmente: fetch lo calcula solo
82     const formData = new FormData();
83     formData.append("imagen", archivo);
84     formData.append("nombre", archivo.name);
85
86     try {
87         const respuesta = await fetch("/api/subir", {
88             method: "POST",
89             body:    formData
90             // IMPORTANTE: NO poner headers Content-Type aqui
91             // FormData necesita que el navegador lo calcule automaticamente
92         });
93
94         // fetch NO lanza error en 4xx/5xx: hay que verificar .ok
95         if (!respuesta.ok) {
96             const error = await respuesta.json();
97             throw new Error(error.error || "HTTP " + respuesta.status);
98         }
99
100        const datos = await respuesta.json();
101        mostrar(
102            "Subida exitosa!\n" +
103            "Guardado como: " + datos.nombre + "\n" +
104            "Tamano:         " + (datos.tamano / 1024).toFixed(1) + " KB\n" +
105            "Tipo:           " + datos.tipo,
106            "ok"
107        );
108    } catch (e) {
109        mostrar("Error al subir: " + e.message, "error");
110    } finally {
111        btnSubir.textContent = "Subir al servidor";
112        btnSubir.disabled    = false;
```

```

113     }
114   }
115
116   function mostrar(texto, tipo) {
117     info.textContent = texto;
118     info.className  = tipo || "";
119   }
120   </script>
121 </body>
122 </html>

```

IntelliJ IDEA — Paso 14: Levantar el servidor y probar

```
node server.js
```

Abre <http://localhost:3000> y:

1. Haz clic en *Seleccionar archivo* y elige una imagen de tu ordenador.
2. La imagen aparece como previsualizacion **antes** de subirla (usando `URL.createObjectURL`).
3. Haz clic en **Subir al servidor**. La respuesta muestra el nombre y tamaño del archivo guardado.
4. Verifica en IntelliJ que la imagen aparecio en la carpeta `subidas/`.

4. Resumen y buenas practicas

4.1. Cuando usar cada mecanismo de almacenamiento

Necesidad	Usar	Por que
Guardar tema oscuro / idioma	localStorage	Persiste indefinidamente, facil API
Carrito de compras (sin cuenta)	localStorage	Persiste entre sesiones
Datos de formulario multipaso	sessionStorage	Se limpia solo al cerrar pestana
Catalogo de productos offline	IndexedDB	Gran volumen, busquedas eficientes
Token JWT de autenticacion	Cookie HttpOnly	JS no puede leerla (protege contra XSS)

4.2. Reglas de oro

1. **Nunca guardes datos sensibles en `localStorage`.** Tokens de autenticacion, claves API y contraseñas deben ir en cookies `HttpOnly`.
2. **Siempre usa `try/catch` con `localStorage`.** Captura el error `QuotaExceededError` si el disco del usuario esta lleno.
3. **Serializa objetos con `JSON.stringify/parse`.** `localStorage` solo acepta strings.
4. **No llares a `video.play()` sin gesto del usuario.** Los navegadores bloquean el autoplay sin una interaccion previa (clic, tecla, etc.).
5. **No pongas `Content-Type` al usar `FormData`.** Deja que `fetch()` lo calcule automaticamente con el boundary correcto.
6. **Verifica `respuesta.ok` despues de `fetch`.** La funcion `fetch` no lanza error en codigos HTTP 4xx o 5xx; debes verificarlo manualmente.
7. **Geolocation requiere HTTPS en produccion.** En localhost funciona sin HTTPS solo para desarrollo.

Guia paso a paso para neofitos · JavaScript 2025
Compilado con L^AT_EX pdflatex guia_almacenamiento_multimedia.tex